Predicting Coronary Heart Disease from BRFSS Survey Answers

Claudia Berra, Paolo Cermelli, Gaia Di Paolo
CS-433 Machine Learning, EPFL, Switzerland

***Abstract*—We predict whether a person reports a heart attack or coronary heart disease (MICHD) from 321 answers to the 2015 BRFSS health survey, where only 8.8% of 328,135 people are positive. We encode the survey column by column, compare 13 methods, and submit an ensemble of two Newton logistic regressions, one with feature products and one followed by boosted trees: F1 = 0.4356 in 5-fold cross-validation, 0.446 on AIcrowd. An ablation shows that the threshold and the encoding matter more than the model. We also find, and keep out, a column that reveals the label.**

I. Introduction

The BRFSS is a yearly US telephone health survey [1]. From its 2015 answers (328,135 training and 109,379 test people) we predict MICHD: the person was told they had a heart attack, angina or coronary heart disease.

With 8.8% positives, answering "healthy" for everybody is 91% accurate, so accuracy is useless. The challenge uses F1, the harmonic mean of *precision* (the share of flagged people who are positive) and *recall* (the share of positive people we flag). Flagging everybody gives F1 = 0.1623, the best rule on one question (self-rated general health) 0.3088.

We contribute a survey-aware pipeline with an ablation of its steps, 13 methods compared on the same folds, an ensemble that beats the best single model in every fold, and a measured label leak.

II. Models and Methods

A. Data and preparation

Our exploratory analysis (`experiments/EDA.md`) found three problems. *Codes are not values:* 7 usually means "don't know", but "retired" for the employment question, so we wrote one rule per column from the codebook. *Missing values are everywhere and informative:* an average column is 47% missing, because of optional modules and of skip logic (a follow-up question is not asked when it does not apply); people who answered the diabetes module have 21.9% MICHD, the others 7.9%. *Many columns repeat others.* Train and test are distributed alike.

Our pipeline fits every statistic (medians, categories, informative missing values) on the training rows of each fold only. *Clean:* special codes become "missing" or 0, each answer gets one unit, and skipped questions are filled with "no". *Select:* drop 41 columns that do not describe the person (ids, dates, survey weights) and the 30 duplicates, then keep 124 of the 236 left with four rules. The main one drops 89 columns whose *mutual information* with the label is below 1 millibit, i.e. knowing them removes almost none of the 431 millibits of uncertainty about the label; the others mostly keep one column per small optional module. *Fill:* add an "is missing" column per group of columns missing together when missingness shifts the MICHD rate by $\geq$3 points; impute the median (numbers) or the mode (yes/no, categories). *Encode:* yes/no becomes 0/1; categories, ordinal answers whose risk is not monotone (e.g. smoking status) and the binned days of poor physical health become one 0/1 column per value with at least 500 people (*one-hot*); then every column is standardized. This gives 336 inputs.

B. Models

Required methods (`implementations.py`): linear regression by gradient descent (GD) and by stochastic GD, least squares, ridge regression, and logistic regression by GD with and without the penalty $\lambda\|w\|^2$. Linear models are fitted to labels ± 1. Least squares uses a pseudo-inverse from the eigendecomposition of $X^\top X$, which is singular when columns are collinear.

Other optimizers of the regularized logistic loss: adaptive-step GD (Barzilai–Borwein [2]), UniXGrad [3], Sophia [4], and *Newton's method*, which uses the curvature of the loss to set each step: about 10 steps reach the minimum, with no step size.

Trees on 32 bins per column [5]: a random forest [6] and gradient boosting [7], i.e. small trees fitted one after the other to the errors left by the previous ones (split gain of XGBoost [8], half of the rows and columns per tree [9]).

Logistic boosting fits the logistic regression with Newton ($\lambda = 10^{-4}$), then starts the boosting from its log-odds $x^\top w$ instead of a constant, like the base margin of XGBoost. The linear part captures the additive effects (age, general health) exactly; 400 trees (depth 3, learning rate 0.05, $\geq$100 people per leaf) learn what is left. It is a simple combination, judged by the ablation.

Newton with expanded features adds squares of the non-binary columns, $\log(1 + x - \min x)$ of the continuous and count ones, and the products of all pairs of the 20 columns most correlated with the label (190 products, chosen inside each fold): 619 columns, $\lambda = 10^{-6}$.

Table I
F1 IN 5-FOLD CV (MEAN ± STD), BASIC PREPARATION (CODES TO MISSING, >50%-MISSING COLUMNS DROPPED, MEDIAN, ONE-HOT) VS. OURS. †WEIGHT AND THRESHOLD SET ON THE OTHER FOLDS; SO EVALUATED, ITS PARTS SCORE 0.4336, 0.4338.

Method	Basic	Ours	Setting (ours)
Linear regr., GD	0.4184	0.4211 ±0.0018	γ 0.1, 300 it.
Linear regr., SGD	0.3996	0.4084 ±0.0028	γ 10^{-4}, $5{\cdot}10^{4}$ it.
Least squares	0.4179	0.4212 ±0.0025	–
Ridge regression	0.4181	0.4216 ±0.0026	λ 10^{-3}
Logistic regr., GD	**0.4270**	0.4311 ±0.0017	γ 0.5, 500 it.
Reg. logistic, GD	**0.4270**	0.4308 ±0.0013	λ 10^{-4}, γ 0.3
Reg. logistic, Newton	–	0.4323 ±0.0020	λ 10^{-4}, ≤20 it.
Reg. logistic, adaptive GD	–	0.4322 ±0.0019	λ 10^{-4}, 300 it.
Reg. logistic, UniXGrad	–	0.4318 ±0.0016	λ 10^{-4}, 200 it.
Reg. logistic, Sophia	–	0.3928 ±0.0050	γ 0.02, 3000 it.
Random forest	–	0.4158 ±0.0018	100 trees, depth 14
Gradient boosting	–	0.4298 ±0.0024	200 trees, depth 3
Logistic boosting	–	0.4338 ±0.0020	400 trees, depth 3
Newton, expanded features	–	0.4339 ±0.0016	619 columns
Ensemble (submitted)†	–	**0.4356** ±0.0020	$a = 0.35$

Ensemble and threshold. The submission averages the two probabilities, $p = a\,p_{\text{Newton}} + (1-a)\,p_{\text{boosting}}$ with $a = 0.35$, and predicts MICHD when $p \geq t$. With 8.8% positives, $t = 0.5$ flags almost nobody, so t maximizes F1 on training predictions, never on the scored rows ($t = 0.206$).

C. Evaluation

All experiments share 5 stratified folds (seed 1): pipeline, model and t are fitted on four folds, F1 is measured on the fifth. We report mean ± standard deviation (about 0.002, so smaller differences are noise) and in how many folds a variant beats its reference. Hyperparameters were searched in stages on small grids. The ensemble weight $a \in \{0, 0.05, \ldots, 1\}$ and t are chosen on out-of-fold probabilities; both models are then refitted on all training rows. `python run.py` rebuilds the submitted file byte for byte in about 11.5 minutes.

III. RESULTS

Baselines (Table I). Our preparation improves each required method by 0.0027 to 0.0088. Logistic beats linear regression by about 0.01; three optimizers of the same loss end within 0.0005, while Sophia, the only stochastic one, stays at 0.3928. Trees alone do worse.

Ablation (Fig. 1). The threshold dwarfs everything: at $t = 0.5$, logistic boosting scores 0.2395 instead of 0.4338. Among data steps only one-hot encoding is large; the selection adds nothing measurable, and each cleaning step not shown changes F1 by less than 0.001. Squares and logarithms change nothing (0.4323 to 0.4324), products do not help logistic boosting (-0.0005), and clinical features (counts of conditions, age × sex, …) lower F1 by 0.0004 to 0.0013.

Submissions (Table II). The ensemble weight is stable across folds (0.35 to 0.40). Our best online score, 0.452, used the leaking column discussed below: the test performance of our submission is 0.446.

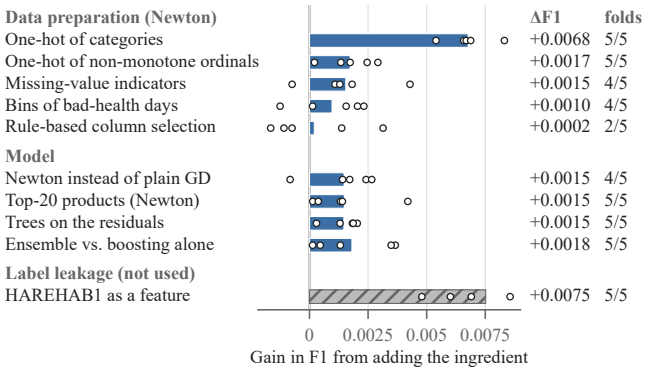


Figure 1. Gain in F1 from adding each ingredient, all else fixed (bars: mean of the 5 folds; dots: each fold; "folds": folds with a gain). Data rows use Newton. Not shown: tuning t instead of 0.5 adds 0.1943.

Table II
PREDICTIONS SENT TO AICROWD. "RULE": PEOPLE ASKED HAREHAB1 ARE SET POSITIVE.

Prediction	HAREHAB1	CV F1	AIcrowd
Ensemble (submitted)	no	**0.4356** ±0.0020	**0.446**
Logistic boosting	no	0.4338 ±0.0020	0.445
Ensemble + rule	rule	0.4414 ±0.0021	0.449
Logistic boosting	feature	0.4413 ±0.0032	0.452

IV. DISCUSSION

Where the gain comes from. All reasonable models end between 0.43 and 0.435: the limit is probably the information in the answers, not the model. Products and trees give Newton the same +0.0015 and their gains do not combine, so they likely capture the same interactions. We did not check that the ensemble's parts err on different people, or that GD lags Newton by not converging.

A column that reveals the label. HAREHAB1 asks about rehabilitation *after a heart attack*, so it is asked only to people who had one: all 773 training rows where it was asked are positive. As a feature it adds 0.0075 in all folds, and as a rule it lifts the ensemble to 0.4414. We do not use it: it reveals the label, and it exists only after the event.

Limitations. The selection rules saw the labels of all training rows, so CV is slightly optimistic (unmeasured; the selection itself adds only 0.0002). The data ablation uses Newton, not the ensemble; $\lambda = 10^{-6}$ is at its grid edge; and there is no nested CV.

V. SUMMARY

Careful encoding and an ensemble of two Newton logistic regressions give F1 = 0.4356 in CV and 0.446 on AIcrowd. Threshold and encoding matter most, and a leaking column would add score, not usefulness.

REFERENCES

[1] Centers for Disease Control and Prevention, "Behavioral risk factor surveillance system: 2015 BRFSS survey data and documentation," https://www.cdc.gov/brfss/annual_data/annual_2015.html, 2016.

[2] J. Barzilai and J. M. Borwein, "Two-point step size gradient methods," *IMA Journal of Numerical Analysis*, vol. 8, no. 1, pp. 141–148, 1988.

[3] A. Kavis, K. Y. Levy, F. Bach, and V. Cevher, "UniXGrad: A universal, adaptive algorithm with optimal guarantees for constrained optimization," in *Advances in Neural Information Processing Systems 32*, 2019.

[4] H. Liu, Z. Li, D. Hall, P. Liang, and T. Ma, "Sophia: A scalable stochastic second-order optimizer for language model pre-training," arXiv:2305.14342, 2023.

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[9] J. H. Friedman, "Stochastic gradient boosting," *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.